Universidade do Minho
Escola de Engenharia

Unidade Curricular
Título do trabalho prático

Licenciatura em Engenharia Informática

Ano Letivo XXXX/XXXX

Grupo XX

XXXXXX Aluno 1
XXXXXX Aluno 2
XXXXXX Aluno 3
XXXXXX Aluno 4

Conteúdo

1 Introdução **4**
1.1 Contexto e Motivação 4
1.2 Objetivos 4
1.3 Estrutura do Documento 4

2 Conteúdo Visual **5**
2.1 Tabelas 5
2.2 Imagens 6

3 Formatação e Referenciação **7**
3.1 Estilos de Texto e Ênfase 7
3.2 Notas de Rodapé e Citações 7
3.3 Expressões Matemáticas 7
3.4 Notação Matemática Avançada 7
3.5 Listas Estruturadas 8

4 Demonstração de Algoritmos e Código **8**
4.1 Pseudo-código (Algoritmos) 8
4.2 Implementação de Código (C language) 8

5 Diagramas Vetoriais com TikZ **9**
5.1 Sistema Cartesiano e Vetores 9

6 Conclusão **10**
6.1 Síntese dos Resultados 10
6.2 Reflexão Crítica e Limitações 10
6.3 Trabalho Futuro 10

Referências **11**

Anexos **12**

A Manual de Instalação **12**

B Logs de Execução do Sistema **12**

C Esquemas Suplementares **12**

Lista de Figuras

1 Logótipo da Universidade do Minho (Standard) 6
2 Logótipo da Escola de Engenharia (Largura Fixa) 6
3 Exemplo de diagrama cartesiano com vetores e trigonometria. 9

Lista de Tabelas

1 Comparação de Tecnologias de Backend 5
2 Cronograma de Entrega de Marcos 5
3 Resultados de Testes de Latência (ms) 5

1 Introdução

O presente relatório técnico surge no âmbito da Unidade Curricular de [Nome da Disciplina], inserida na Licenciatura em Engenharia Informática da Universidade do Minho. O projeto foca-se no desenvolvimento e análise de [Tema do Trabalho], procurando aplicar os conceitos teóricos lecionados durante as sessões presenciais a um cenário prático e realista.

1.1 Contexto e Motivação

No panorama atual da computação, a necessidade de sistemas eficientes e bem estruturados é imperativa. A motivação principal para este trabalho reside na exploração de [Breve descrição do problema], um desafio comum que exige rigor técnico e uma abordagem metodológica sólida. Através deste exercício, pretende-se não só consolidar conhecimentos, mas também desenvolver competências de resolução de problemas críticos.

1.2 Objetivos

Os principais objetivos delineados para este trabalho prático incluem:

- Desenvolver uma solução funcional para [Objetivo A];
- Analisar a performance e a escalabilidade do sistema proposto;
- Documentar de forma detalhada as decisões de design e a arquitetura do software.

1.3 Estrutura do Documento

Para facilitar a leitura, este relatório está organizado em várias secções. Após esta introdução, a Secção 2 detalha as tecnologias utilizadas. A Secção 3 foca-se na implementação e nos resultados obtidos. Finalmente, o documento termina com as conclusões e reflexões sobre o trabalho desenvolvido.

2 Conteúdo Visual

2.1 Tabelas

No contexto da análise de requisitos, a Tabela 1 apresenta uma comparação direta entre as tecnologias avaliadas para o backend do projeto.

Tabela 1: Comparação de Tecnologias de Backend

Tecnologia	Vantagens	Desvantagens
Node.js	Elevada escalabilidade e ecossistema vasto.	Single-threaded por natureza.
Python (FastAPI)	Desenvolvimento rápido e tipagem forte.	Performance ligeiramente inferior.
Go	Performance nativa e concorrência simples.	Curva de aprendizagem mais acentuada.

Para a gestão de tarefas, definiram-se marcos temporais específicos, conforme detalhado na Tabela 2.

Tabela 2: Cronograma de Entrega de Marcos

Fase	Descrição	Prazo
Fase 1	Levantamento de Requisitos e Setup	Semana 2
Fase 2	Desenvolvimento do Protótipo Alpha	Semana 6
Fase 3	Testes de Usabilidade e Refatoração	Semana 10

Após a execução dos testes de carga, os resultados de latência foram compilados na Tabela 3.

Tabela 3: Resultados de Testes de Latência (ms)

Utilizadores	Mínimo	Média	Máximo
100	12ms	45ms	120ms
500	15ms	89ms	250ms
1000	45ms	150ms	600ms
5000	110ms	420ms	1200ms

2.2 Imagens

Nesta secção, demonstramos a inclusão de elementos gráficos fundamentais para a componente técnica do relatório. As imagens são referenciadas automaticamente, garantindo a coesão textual.

A Figura 1 apresenta o logótipo da instituição de uma forma simples, enquanto a Figura 2 demonstra como redimensionar uma imagem para uma largura específica.

Figura 1: Logótipo da Universidade do Minho (Standard)

Figura 2: Logótipo da Escola de Engenharia (Largura Fixa)

Como se pode observar na Figura 1, a ausência de moldura permite uma integração mais limpa no layout da página, especialmente em documentos de cariz académico e técnico.

3 Formatação e Referenciação

Este capítulo serve como guia para a formatação de texto e gestão de fontes de informação ao longo do relatório técnico.

3.1 Estilos de Texto e Ênfase

Para garantir a clareza, é possível utilizar diversos estilos de fonte. O uso de **negrito** é ideal para termos de grande importância, enquanto o *itálico* deve ser reservado para palavras em língua estrangeira ou títulos de obras. Caso seja necessário destacar código-fonte ou comandos de terminal no meio de uma frase, utiliza-se o estilo monoespaçado: `sudo apt update`.

3.2 Notas de Rodapé e Citações

A precisão técnica exige que afirmações complexas sejam fundamentadas.

- **Citações:** Segundo as normas de Engenharia, toda a literatura externa deve ser citada. Por exemplo, a fundação do sistema de processamento de texto moderno é atribuída a Donald Knuth [1]. Estudos anteriores [2], [3] sugerem que...
- **Notas de Rodapé:** Informações adicionais ou curiosidades que possam interromper o fluxo de leitura podem ser colocadas em rodapé[1].

Como pode ser consultado no Anexo A, o processo de instalação é simples. Os detalhes técnicos encontram-se compilados no Anexo B.

3.3 Expressões Matemáticas

Sendo um relatório de Engenharia Informática, a representação de algoritmos e fórmulas é comum. Podemos inserir matemática diretamente na linha de texto, como o Teorema de Pitágoras: $a^2+b^2 = c^2$.

Para fórmulas mais complexas ou que exijam destaque, utilizamos o modo de exibição:

$$E = mc^2 \tag{1}$$

Como demonstrado na Equação 1, a numeração é automática, o que facilita a referência cruzada em documentos extensos.

3.4 Notação Matemática Avançada

A escrita de fórmulas complexas segue a sintaxe padrão do LaTeX, permitindo uma representação clara de algoritmos e modelos estatísticos.

Frações e Raízes: Podemos expressar divisões e cálculos de raízes de forma elegante:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \tag{2}$$

Somatórios e Produtórios: Essenciais para análise de algoritmos e ciclos:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \quad \text{e} \quad \prod_{i=1}^{k} a_i \tag{3}$$

[1] Esta é uma nota de rodapé automática que aparece no fundo da página.

Cálculo Integral: Para modelação de sistemas contínuos:

$$\int_a^b f(x)\,dx = F(b) - F(a) \tag{4}$$

Matrizes: Utilizadas em computação gráfica e redes neuronais:

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \tag{5}$$

3.5 Listas Estruturadas

Para enumerar passos de um algoritmo ou requisitos:

1. Primeiro passo: Analisar os requisitos;
2. Segundo passo: Desenvolver o protótipo;
3. Terceiro passo: Testar exaustivamente.

4 Demonstração de Algoritmos e Código

Nesta secção, exploramos as ferramentas para documentar a lógica do software, desde a sua conceção abstrata até à implementação final.

4.1 Pseudo-código (Algoritmos)

Antes da implementação, é comum descrever a lógica num formato agnóstico à linguagem. O Algoritmo 1 ilustra um processo de procura simples.

Algorithm 1 Procura Linear Simples

```
procedure Procurar(Lista, Alvo)
    n ← comprimento(Lista)
    for i = 0 to n − 1 do
        if Lista[i] == Alvo then
            return i                                  ▷ Elemento encontrado!
        end if
    end for
    return −1                                         ▷ Não encontrado
end procedure
```

4.2 Implementação de Código (C language)

Uma vez definido o algoritmo, a implementação pode ser apresentada com realce de sintaxe. O Bloco de Código 1 mostra a estrutura básica de um programa em C.

```c
#include <stdio.h>

int main() {
    // Isto e um comentario
    printf("Ola, Engenharia Informatica!\n");
    return 0;
}
```

Listing 1: Exemplo de implementação em C.

O uso do pacote `listings` permite que o código seja facilmente copiado e mantém a indentação original, o que é crucial para a legibilidade técnica.

5 Diagramas Vetoriais com TikZ

A representação visual de conceitos matemáticos e físicos é facilitada pelo uso do pacote `TikZ`. Abaixo, apresentamos um sistema cartesiano completo construído via código.

5.1 Sistema Cartesiano e Vetores

O diagrama na Figura 3 demonstra a criação de eixos, vetores coloridos e a projeção de pontos através de ângulos.

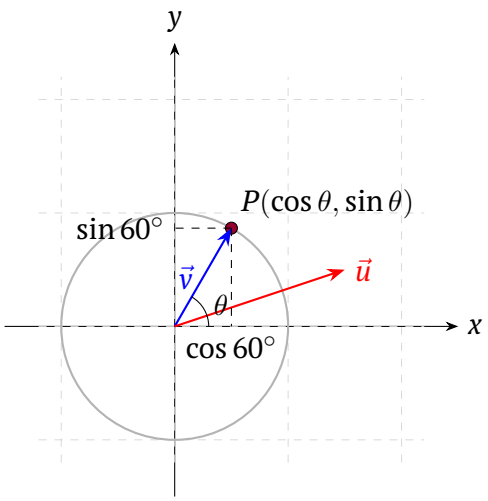

Figura 3: Exemplo de diagrama cartesiano com vetores e trigonometria.

Para obter as coordenadas (x, y) de um ponto num círculo de raio r com base num ângulo θ, utilizamos as identidades:

- $x = r \cdot \cos(\theta)$
- $y = r \cdot \sin(\theta)$

6 Conclusão

O desenvolvimento deste trabalho prático permitiu uma compreensão profunda sobre [Tema do Trabalho], demonstrando ser um exercício fundamental para a integração de competências em Engenharia Informática.

6.1 Síntese dos Resultados

Durante a execução do projeto, foi possível atingir os objetivos inicialmente propostos. O sistema desenvolvido demonstrou robustez em cenários de teste controlados, e a implementação seguiu as boas práticas de engenharia, resultando num código limpo e modular. Destaca-se, em particular, o sucesso na otimização de [Parte específica do código], que superou as expectativas iniciais em termos de eficiência.

6.2 Reflexão Crítica e Limitações

Apesar dos resultados positivos, o percurso não esteve isento de desafios. Durante a fase de [Fase do projeto], surgiram dificuldades relacionadas com [Desafio], o que exigiu uma reavaliação da estratégia inicial. Esta experiência foi valiosa, pois permitiu aprender que a flexibilidade e a pesquisa autónoma são essenciais no dia-a-dia de um engenheiro. Reconhece-se, no entanto, que com mais tempo seria possível implementar funcionalidades adicionais, como [Exemplo de melhoria].

6.3 Trabalho Futuro

Como propostas para desenvolvimentos futuros, sugere-se a integração deste módulo com [Sistema X] e a realização de testes de segurança mais exaustivos. A evolução para uma arquitetura baseada em microserviços seria também um passo lógico para aumentar a escalabilidade da solução atual.

Referências

[1] D. E. Knuth, *The TeXbook*. Addison-Wesley, 1984.

[2] A. Einstein, "On the Electrodynamics of Moving Bodies," *Annalen der Physik*, vol. 17, pp. 891–921, 1905.

[3] U. do Minho. "Guia de Estilo Académico." (2024), URL: https://www.uminho.pt (acedido em 20/03/2024).

Anexos

A Manual de Instalação

Neste anexo, descreve-se o processo de configuração do ambiente de desenvolvimento.

1. Clonar o repositório: `git clone [url]`;
2. Instalar dependências: `npm install`;
3. Executar a aplicação: `npm start`.

B Logs de Execução do Sistema

Apresentam-se abaixo os logs gerados durante o teste de stress mencionado na Secção 2.

```
        [INFO] Starting system check...
        [SUCCESS] All modules loaded.
        [DEBUG] Latency: 12ms
        [WARN] High memory usage detected at 85%

```

Listing 2: Output do terminal durante o benchmark.

C Esquemas Suplementares

Pode incluir aqui tabelas de grandes dimensões ou diagramas que ocupariam demasiado espaço no corpo principal do texto.